

Лабораторная работа №1

Вариант №27

Задача:

Для таблично заданной функции

x	1.0	1.2	1.5	1.6	1.8	2.0
f(x)	5.000	6.899	11.180	13.133	18.119	25.000

построить сплайн-функцию и использовать ее для нахождения корня уравнения $f(x) = 6x + 3$ на промежутке $[1, 2]$ методом бисекции.

Код:

```
import numpy as np
import pandas as pd
import math
from scipy.interpolate import CubicSpline

x_data = np.array([1.0, 1.2, 1.5, 1.6, 1.8, 2.0])
y_data = np.array([5.000, 6.899, 11.180, 13.133, 18.119, 25.000])

f = CubicSpline(x_data, y_data)

# Создание таблицы
data = {
    'a': [],
    'b': [],
    'x': [],
    'f(x)': []
}
df = pd.DataFrame(data)

def equation(x):
    return f(x) - (6 * x + 3)

def bisection_method(a, b, tol, max_iter=100):
    if equation(a) * equation(b) >= 0:
        raise ValueError("Функция должна иметь разные знаки на концах интервала [a, b].")

    iter_count = 0
    while (b - a) / 2 > tol and iter_count < max_iter:
        c = (a + b) / 2
        if equation(c) == 0:
            return c
        elif equation(a) * equation(c) < 0:
            b = c
        else:
            a = c
        iter_count += 1
        new_row = {'a': a, 'b': b, 'x': c, 'f(x)': equation(c)}
        df.loc[len(df)] = new_row

    return (a + b) / 2

tol = 1e-6
a = 1.0
b = 2.0
```

```
try:
    root = bisection_method(a, b, tol)
    print(df)
    digit = (-1) * math.log10(tol)
    print(f"Найденный корень: {round(root, int(digit))}")
except ValueError as e:
    print(e)
```

Объяснение:

Для решения данной задачи были использованы библиотеки NumPy, предназначенная для работы с массивами и линейной алгеброй, и SciPy, в данном случае для интерполяции данных (см. 4 строка кода - `scipy.interpolate`).

Использован метод CubicSpline, который интерполирует данные с помощью кубических полиномов 3-й степени. Метод дает достаточно точный и гладкий результат.

Был реализован метод бисекции `bisection_method()`, принимающий значения отрезка $[a, b]$, нужную нам точность `tol` и ограничение в виде максимального количества итераций `max_iter`. Метод заключается в делении на отрезки область решения функции до тех пор, пока мы не приблизимся к решению с нужной нам точностью. Метод легкий в вычислениях, но не самый быстрый, т.к. чем большая точность нам нужна, тем дольше придется считать.

`equation(a) * equation(b) >= 0` - проверка одного ли знака значения функции, если разного, то вычисляем среднее арифметическое между двумя точками $c = (a + b) / 2$

Если функция от среднего арифметического равна нулю

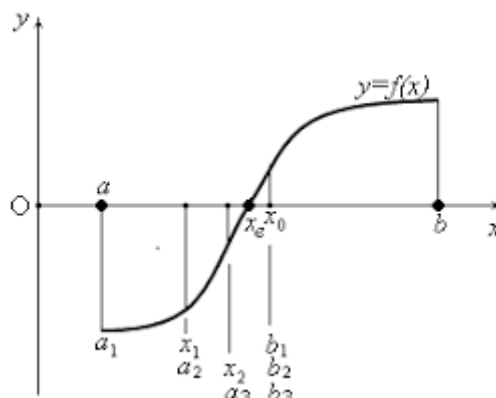
`equation(c) == 0`,

то c - это корень уравнения, если

`equation(a) * equation(c) < 0`,

то корень лежит в интервале $[a, c]$, иначе корень лежит на интервале $[c, b]$.

Повторяем данный алгоритм пока не достигнем нужной нам точности `tol`



Вывод программы:

	a	b	x	f(x)
0	1.500000	2.000000	1.500000	-0.820000
1	1.500000	1.750000	1.750000	3.216601
2	1.500000	1.625000	1.625000	0.921655
3	1.562500	1.625000	1.562500	-0.010942
4	1.562500	1.593750	1.593750	0.439146
5	1.562500	1.578125	1.578125	0.210150
6	1.562500	1.570312	1.570312	0.098627
7	1.562500	1.566406	1.566406	0.043600
8	1.562500	1.564453	1.564453	0.016269
9	1.562500	1.563477	1.563477	0.002648
10	1.562988	1.563477	1.562988	-0.004151
11	1.563232	1.563477	1.563232	-0.000752
12	1.563232	1.563354	1.563354	0.000948
13	1.563232	1.563293	1.563293	0.000098
14	1.563263	1.563293	1.563263	-0.000327
15	1.563278	1.563293	1.563278	-0.000115
16	1.563286	1.563293	1.563286	-0.000009
17	1.563286	1.563290	1.563290	0.000045
18	1.563286	1.563288	1.563288	0.000018
Найденный корень: 1.563287				

Заключение:

Использованный метод CubicSpline хорошо справляется в данной постановке задачи и дает высокую точность интерполяции, однако, если есть большой разброс между точками или шум, то сплайн становится неестественным и дает не удовлетворительные результаты.

Метод бисекции — это надежный и простой в реализации метод для нахождения корней уравнений, но его медленная сходимость и ограниченные условия применимости делают его менее эффективным для сложных задач. Однако для базовых задач, где важна надежность и простота, он остается отличным выбором